1. Problem Title & Source

Title: **Consecutive Prime Sum**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

You are given a number **N**.

Your task is to determine whether **N can be represented as the sum of consecutive prime numbers**.

Rules:

1. Only **prime numbers** are allowed.
2. The primes must be **consecutive in the prime sequence**.
3. At least **two consecutive primes** must be used.

If the number **N can be expressed as the sum of consecutive prime numbers**, print:

YES

Otherwise print:

NO

3. Examples

Example 1

Input

41

Output

YES

Explanation

Prime numbers sequence:

2, 3, 5, 7, 11, 13, 17, 19, ...

We can express:

2 + 3 + 5 + 7 + 11 + 13 = 41

Since the sum of consecutive primes equals **41**, the answer is **YES**.

Example 2

Input

20

Output

NO

Explanation

There is **no sequence of consecutive prime numbers** whose sum equals **20**.

4. Constraints

1 ≤ N ≤ 10^4

Observations

- We first need **all primes ≤ N**
- Then check **consecutive prime sums**

5. Core Concept (Pattern)

This problem combines:

Prime number generation

Prefix sum / sliding window

6. Thought Process

Step 1 — Generate Prime Numbers

We generate all primes up to **N** using:

Sieve of Eratosthenes

Example primes ≤ 20:

2, 3, 5, 7, 11, 13, 17, 19

Step 2 — Check Consecutive Prime Sums

We try all consecutive sequences:

Example

2 + 3 = 5

2 + 3 + 5 = 10

2 + 3 + 5 + 7 = 17

2 + 3 + 5 + 7 + 11 = 28

If any sequence equals **N**, answer is **YES**.

7. Visualization

Example

N = 41

Prime list

2 3 5 7 11 13 17

Consecutive sums

Sequence	Sum
2 + 3	5
2 + 3 + 5	10

2 + 3 + 5 + 7	17
2 + 3 + 5 + 7 + 11	28
2 + 3 + 5 + 7 + 11 + 13	41

Match found.

8. Pseudocode

```
read N

generate primes ≤ N

for i from 0 to size(primes)

   sum = 0

   for j from i to size(primes)

      sum += primes[j]

      if sum == N AND length ≥ 2
         print YES
         stop

      if sum > N
         break

print NO
```

9. Code Implementation

Python

```
import math

n = int(input())

# generate primes
primes = []
```

```
for num in range(2, n+1):
    prime = True
    for i in range(2, int(math.sqrt(num))+1):
        if num % i == 0:
            prime = False
            break
    if prime:
        primes.append(num)

# check consecutive sum
for i in range(len(primes)):
    s = 0
    count = 0
    for j in range(i, len(primes)):
        s += primes[j]
        count += 1

        if s == n and count >= 2:
            print("YES")
            exit()

        if s > n:
            break

print("NO")
```

Java

```
import java.util.*;

class Main {

    static boolean isPrime(int n){

        if(n < 2) return false;

        for(int i=2;i*i<=n;i++)
            if(n%i==0)
```

```
            return false;

        return true;
    }

    public static void main(String[] args){

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        List<Integer> primes = new ArrayList<>();

        for(int i=2;i<=n;i++)
            if(isPrime(i))
                primes.add(i);

        for(int i=0;i<primes.size();i++){

            int sum = 0;
            int count = 0;

            for(int j=i;j<primes.size();j++){

                sum += primes.get(j);
                count++;

                if(sum == n && count >= 2){
                    System.out.println("YES");
                    return;
                }

                if(sum > n)
                    break;
            }
        }
```

```
        System.out.println("NO");
    }
}
```

10. Time & Space Complexity

Time Complexity

Prime generation

$O(N \sqrt{N})$

Consecutive sum check

$O(P^2)$

Where

P = number of primes $\leq N$

Since $\mathbf{N \leq 10^4}$, this is efficient.

Space Complexity

$O(P)$

For storing primes.

11. Common Mistakes

1. Forgetting that **at least two primes must be used**
2. Generating primes incorrectly
3. Not stopping when sum exceeds N
4. Considering non-consecutive primes

12. Detailed Dry Run

Input

N = 20

Primes

2 3 5 7 11 13 17 19

Check sequences

2+3 = 5

2+3+5 = 10

2+3+5+7 = 17

2+3+5+7+11 = 28 > 20

Next

3+5+7 = 15

3+5+7+11 = 26

No sequence gives **20**.

Output

NO

13. Real-Life Applications

Prime sequences appear in:

- Cryptography
- Number theory
- Data hashing
- Mathematical modeling

14. Common Traps

1. Checking only two primes
2. Forgetting **consecutive requirement**
3. Inefficient prime generation
4. Not handling small values of N

15. Related Problems

Prime / sequence problems:

LC 204 Count Primes

LC 1175 Prime Arrangements

LC 2523 Closest Prime Numbers in Range

16. Alternate Approaches

Approach	Complexity	Notes
Brute prime checking	$O(N^2)$	slow
Prime list + consecutive sums	$O(P^2)$	optimal

17. Why This Solution Works

By generating all primes up to **N**, we reduce the problem to checking **consecutive subsequences of primes**. Since sums grow quickly, the loop terminates early once the sum exceeds **N**, making the approach efficient.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Count **how many ways** N can be expressed
2. Find **longest consecutive prime sum ≤ N**

3. Use **sliding window instead of nested loops**
4. Prime sum problems with **large N**